

Relazione dell'elaborato per il corso "Algoritmi e Strutture Dati"

Applicazione del paradigma della progettazione dinamica al problema di ricerca della sottosequenza comune di lunghezza massima fra due stringhe/sequenze
(Longest Common Subsequence)

Saleri Giorgio, Viola Luca

Indice

Presentazione del problema (LCS)	3
1. Applicabilità della programmazione dinamica	3
1.1. Caratterizzazione della struttura di una soluzione ottima	3
1.2. Definizione ricorsiva del valore di una soluzione ottima	4
1.3. Calcolo del valore di una soluzione ottima	5
1.4. Costruzione della soluzione ottima	6
2. Implementazione	8
2.1. Misurazioni	9
2.2. Visualizzazione dei dati	14
3. Varianti dell'algoritmo LCS-LENGTH	17
3.1. Misurazioni	19
4. Note	21
5. Bibliografia:	21

Presentazione del problema (LCS)

Il problema della massima sottosequenza comune (*longest common subsequence*) consiste nel trovare la sottosequenza di lunghezza massima comune a due stringhe di input.

Una sottosequenza di una stringa è una nuova stringa generata dall'originale, alla quale sono stati rimossi zero o più elementi, mantenendo però l'ordine relativo dei caratteri rimanenti.

Notazione: d'ora in avanti il problema della massima sottosequenza comune verrà indicato univocamente con la sigla LCS al fine di ridurre la verbosità del documento.

Definizione di sottosequenza:

Data la stringa $X = \langle x_1, x_2, \dots, x_n \rangle$, un'altra stringa $Z = \langle z_1, z_2, \dots, z_k \rangle$ è detta sottosequenza di X se $\exists \langle i_1, i_2, \dots, i_k \rangle$ di indici di X tale che $\forall j = 1, 2, \dots, k$ si ha $x_{i_j} = z_j$.

Definizione di sottosequenza comune:

Date due sequenze X e Y , diciamo che una sequenza Z è sottosequenza comune di X e Y se Z è una sottosequenza di X ed Y .

Definizione del problema:

Date due stringhe $X = \langle x_1, x_2, \dots, x_m \rangle$ e $Y = \langle y_1, y_2, \dots, y_n \rangle$, trovare la sottosequenza di lunghezza maggiore $Z = \langle z_1, z_2, \dots, z_k \rangle$ comune a X e Y .

Applicazioni dell'algoritmo

- Grado di parentela tra due specie: si analizzano sequenze di DNA delle due specie con LCS per controllare quanto queste siano simili, e quindi determinarne il legame di parentela delle specie;
- Sistemi di controllo di versione: in *git* viene utilizzato il principio alla base di LCS per la gestione dei merge;
- Correzione ortografica: viene sfruttato LCS per facilitare il calcolo della distanza di edit utilizzata per i correttori automatici.

1. Applicabilità della programmazione dinamica

1.1. Caratterizzazione della struttura di una soluzione ottima

La soluzione più ingenua al problema è quella di enumerare tutte le sottosequenze della prima stringa X e controllare singolarmente se queste sono anche sottosequenze della seconda stringa Y , tenendo nel frattempo traccia della sottosequenza comune più lunga trovata.

Una stringa X composta da m caratteri ha al suo interno 2^m possibili sottosequenze (ogni sottosequenza corrisponde ad un sottoinsieme degli indici di X), perciò questo approccio richiede un tempo esponenziale e non è applicabile al problema in modo generale.

Per caratterizzare matematicamente la soluzione è comodo definire il concetto di "prefisso": data la sequenza $X = \langle x_1, x_2, \dots, x_m \rangle$, l' **i -esimo prefisso di X** è la sottosequenza $X_i = \langle x_1, x_2, \dots, x_i \rangle$. Quindi l' i -esimo prefisso è la sottosequenza che include i primi i elementi di X (X_0 è la stringa vuota).

Questo problema gode della proprietà della sottostruttura ottima: le classi di sottoproblemi si possono rappresentare come coppie di prefissi delle due sequenze di input. Il problema complessivo è riducibile in sottoproblemi partendo dalle stringhe intere ed andando man mano ad analizzare i prefissi sempre più brevi.

Teorema:

Siano $X = \langle x_1, x_2, \dots, x_m \rangle$ e $Y = \langle y_1, y_2, \dots, y_n \rangle$ due sequenze di input e sia $Z = \langle z_1, z_2, \dots, z_k \rangle$ una qualsiasi più lunga sottosequenza comune di X, Y (non è detto che Z sia l'unica sottosequenza comune).

1. Se $x_m = y_n$ allora $z_k = x_m = y_n$ e quindi Z_{k-1} è una più lunga sottosequenza comune di X_{m-1}, Y_{n-1}
2. Se $x_m \neq y_n$ e $z_k \neq x_m$ allora Z è una più lunga sottosequenza comune di X_{m-1}, Y
3. Se $x_m \neq y_n$ e $z_k \neq y_n$ allora Z è una più lunga sottosequenza comune di X, Y_{n-1}

Questo teorema è fondamentale per procedere con la definizione ricorsiva di una soluzione e spiega come si può scomporre il problema in problemi di dimensione minori garantendo che le loro soluzioni siano utili per costruire la soluzione ottima del problema originale.

Una LCS di due stringhe contiene al suo interno una LCS di certi prefissi delle due stringhe (ovvero il problema gode della proprietà della sottostruttura ottima e quindi si potrà costruire la LCS a partire da sottosequenze comuni minori).

1.2. Definizione ricorsiva del valore di una soluzione ottima

Il teorema ci indica che ci sono uno o due sottoproblemi da analizzare per trovare una più lunga sottosequenza comune di X, Y .

Nel primo caso in cui gli ultimi caratteri delle due stringhe corrispondono ($x_m = y_n$) dobbiamo trovare la LCS di X_{m-1}, Y_{n-1} , ovvero le stringhe senza l'ultimo carattere.

Nel caso in cui invece gli ultimi caratteri non corrispondono si vanno a risolvere due sottoproblemi ovvero trovare la LCS sia di X_{m-1}, Y , che di X, Y_{n-1} .

La più lunga tra le due sottosequenze trovate risolvendo questi problemi è una LCS di X, Y . Questi casi coprono tutte le possibilità e perciò si può concludere che all'interno della LCS di X, Y si deve sicuramente usare una delle soluzioni ottime dei sottoproblemi.

Inoltre questo problema presenta il tipico caso di sottoproblemi sovrapposti, ovvero al fine di trovare la LCS si risolvono più volte gli stessi sottoproblemi (ad esempio se devo trovare la LCS di X, Y_{n-1} andrò a cercare nuovamente la LCS di X_{m-1}, Y_{n-1}). È necessario definire una ricorrenza per il valore di una soluzione ottima affinché il problema sia risolvibile in modo ricorsivo.

Si definisce quindi $c[i, j]$ come la lunghezza di una più lunga sottosequenza comune delle stringhe X_i, Y_j . Se $i = 0, j = 0$, si intende che una delle sequenze ha lunghezza 0 e quindi la LCS è una stringa vuota (anch'essa di lunghezza 0).

Con questa definizione è possibile definire la formula ricorsiva

$$c[i, j] = \begin{cases} 0 & \text{se } i = 0 \text{ o } j = 0 \\ c[i-1, j-1] + 1 & \text{se } i, j > 0 \text{ e } x_i = y_j \\ \max\{c[i, j-1], c[i-1, j]\} & \text{se } i, j > 0 \text{ e } x_i \neq y_j \end{cases}$$

In questa formulazione ricorsiva è racchiuso il caso base per la lunghezza nulla ed i due casi visti prima con il teorema.

1.3. Calcolo del valore di una soluzione ottima

Usando l'equazione sopra definita si potrebbe scrivere facilmente un algoritmo ricorsivo per calcolare la lunghezza della LCS in modo top-down che però richiederebbe un tempo esponenziale vista la ripetizione dei sottoproblemi.

Invece di ripetere i calcoli con tutte le possibili sottosequenze si costruisce una tabella che risolve i sottoproblemi di dimensione minore e li combina man mano per arrivare alla soluzione completa in tempo polinomiale. Il problema complessivo però ha solo $\Theta(m \cdot n)$ sottoproblemi differenti dato che dobbiamo calcolare $c[i, j]$ per $0 \leq i \leq m$, $0 \leq j \leq n$. Si può quindi usare la programmazione dinamica per calcolare le soluzioni in modo bottom-up

LCS-LENGTH(X, Y)

```
1  m = length[X]
2  n = length[Y]
3  Sia c[0 ... m, 0 ... n] una nuova tabella per i valori
4  Sia b[1 ... m, 1 ... n] una nuova tabella per le scelte (direzioni)

5  for i = 0 to m do c[i, 0] = 0    // Inizializzazione dei casi base
6  for j = 0 to n do c[0, j] = 0    // (colonna 0 e riga 0 a zero)

7  for i = 1 to m                    // Riempimento iterativo bottom-up riga per riga
8      do for j = 1 to n
9          do if X[i] == Y[j]
10             then c[i, j] = c[i-1, j-1] + 1
11                 b[i, j] = '↖'          //salva provenienza diagonale
12             else if c[i-1, j] ≥ c[i, j-1]
13                 then c[i, j] = c[i-1, j]
14                     b[i, j] = '↑'      //salva provenienza dall'alto
15             else c[i, j] = c[i, j-1]
16                 b[i, j] = '←'          //salva la provenienza da sinistra
17  return c, b
```

LCS-LENGTH memorizza i valori $c[i, j]$ in una tabella $c[0 \dots m, 0 \dots n]$ e ne calcola gli elementi riga per riga riempiendo la prima riga da sinistra a destra, poi la seconda riga e così via fino all'ultima.

Viene utilizzata anche la tabella $b[1 \dots m, 1 \dots n]$ che semplifica la costruzione della soluzione ottima, $b[i, j]$ punta al valore della tabella che corrisponde alla soluzione ottima del sottoproblema che è stato scelto per calcolare $c[i, j]$ (è la tabella che contiene le frecce). **LCS-LENGTH** al termine restituisce entrambe le tabelle e nella cella $c[m, n]$ si trova la lunghezza di una LCS di X, Y , mentre partendo dalla cella $b[m, n]$ si ricostruisce il percorso per individuare la LCS.

Complessità temporale e spaziale

Il calcolo di ogni elemento della tabella richiede un tempo costante $\Theta(1)$, questa istruzione si trova annidata all'interno di due cicli che vengono eseguiti rispettivamente per m, n iterazioni e perciò il tempo di esecuzione dell'algoritmo **LCS-LENGTH** è proporzionale al numero di iterazioni $T(n) = \Theta(m \cdot n)$.

La complessità spaziale è $\Theta(m \cdot n)$, cioè lo spazio necessario per istanziare le due tabelle in memoria, le singole stringhe passate in input occupano meno spazio delle tabelle che costituiscono l'occupazione di memoria dominante. Essendo un algoritmo iterativo lo spazio in memoria necessario per la pila dei contesti è $\Theta(1)$, quindi non influisce sulla complessità spaziale totale.

1.4. Costruzione della soluzione ottima

Grazie alla tabella b restituita dall'algoritmo si può velocemente costruire una LCS di X, Y . Si inizia da $b[m, n]$ e si attraversa la tabella seguendo le frecce, ogni volta che si incontra una freccia diagonale in $b[i, j]$ allora l'elemento $x_i = y_j$ appartiene alla LCS trovata dall'algoritmo. Così facendo si incontrano gli elementi della LCS in ordine inverso e la si costruisce per intero, dato che a riga 5 la stampa avviene dopo la chiamata ricorsiva, i caratteri della LCS vengono stampati nell'ordine corretto. (prima si esplora la tabella con le chiamate ricorsive incontrando gli elementi in ordine inverso e poi al ritorno li si stampa)

```
PRINT-LCS(b, X, i, j)    // chiamata iniziale: PRINT-LCS(b, X, m, n)
1  if i == 0 or j == 0 return          // la LCS è la stringa vuota
2  if b[i, j] = '\ '          //diagonale
3      PRINT-LCS(b, X, i-1, j-1)
4      stampa X[i]
5  else if b[i, j] = '↑'
6      PRINT-LCS(b, X, i-1, j)
7  else PRINT-LCS(b, X, i, j-1)
```

Complessità temporale

L'algoritmo PRINT-LCS stampa una LCS di X, Y con gli elementi in ordine di indice crescente. La stampa richiede un tempo $O(m + n)$ perchè ad ogni chiamata ricorsiva decrementa almeno uno tra i contatori i, j e quindi le operazioni a tempo costante presenti nell'algoritmo vengono eseguite al più $m + n$ volte.

Complessità spaziale

L'algoritmo richiede $\Theta(m \cdot n)$ per l'istanziazione delle variabili, in particolare della tabella b . Per quanto riguarda lo spazio necessario per la pila dei contesti è necessario calcolare il valore massimo di contesti impilati. La chiamata ricorsiva a riga 3 e succeduta da un'istruzione, quindi non è una ricorsione in coda. Nonostante la presenza di una chiamata ricorsiva non in coda, è necessario differenziare l'analisi quando si utilizza un compilatore ottimizzante e non.

Analisi compilatore non ottimizzante:

Il caso pessimo si riscontra con le chiamate a riga 5 e 7, in questi casi viene ridotto solamente il valore di j oppure i . Il numero massimo di contesti impilati nel caso pessimo è quindi $m + n$. La complessità è uguale a $O(m + n)$.

Analisi compilatore ottimizzante:

Dal punto precedente abbiamo appurato che il caso pessimo si verifica quando vengono chiamate le istruzioni alle righe 5 e 7. Tuttavia queste chiamate sono ricorsive in coda, se si utilizzasse un compilatore ottimizzante la pila dei contesti rimarrebbe costante (ad esempio se le due stringhe X, Y fossero completamente diverse non si hanno frecce diagonali nella tabella b , ed in fase di stampa verrebbero sempre eseguire le chiamate ricorsive in coda, in questo caso ottimo l'algoritmo di stampa occuperebbe un solo contesto).

Il caso pessimo relativo al compilatore ottimizzante invece si riscontra quando viene eseguita la chiamata a riga 3 (mai ottimizzabile) per tutti i valori di i e j .

Quindi in questo caso la complessità spaziale è $O(\min(m, n))$

La complessità spaziale totale dell'algoritmo è

- Compilatore non ottimizzante: $\Theta(m \cdot n) + O(m + n) = \Theta(m \cdot n)$
- Compilatore ottimizzante: $\Theta(m \cdot n) + O(\min(m, n)) = \Theta(m \cdot n)$

2. Implementazione

Abbiamo scelto come linguaggio di programmazione *C++*, un linguaggio compilato e senza garbage collector, che quindi fornisce misurazioni di tempo e memoria meno sporcate dal rumore. Tutto il codice è stato scritto e fatto girare all'interno dell'ambiente di sviluppo *Visual Studio Code*.

Il tempo è stato misurato catturando due istanti di tempo usando la libreria `<chrono>`, il primo subito prima di invocare la funzione interessata ed il secondo appena terminata la sua esecuzione; la loro differenza è la durata che abbiamo convertito in millisecondi.

Per misurare la memoria abbiamo usato due librerie *Windows*:

- `<windows.h>` che fornisce l'accesso al processo corrente `GetCurrentProcess()` e la funzione `EmptyWorkingSet()` per liberare la memoria in modo da effettuare misure pulite.
- `<psapi.h>` che fornisce la struttura `PROCESS_MEMORY_COUNTERS_EX` con i suoi campi tra cui `WorkingSetSize` e la funzione `GetProcessMemoryInfo()` che effettua la lettura vera e propria dei contatori di memoria.

Prima di ogni misurazione il *working set* del processo (la quantità di memoria che la sua esecuzione richiede) viene svuotato tramite `EmptyWorkingSet()` che rimuove dalla memoria fisica le pagine non necessarie, riportando la sua occupazione ad un valore minimo che sarà lo stesso ad ogni misura ripetuta; subito dopo viene letto il valore corrente di `WorkingSetSize` come riferimento iniziale.

A questo punto viene eseguito l'algoritmo e al termine si legge nuovamente il valore `WorkingSetSize`, la differenza tra le due letture rappresenta la memoria che è stata allocata per le strutture dati dell'algoritmo (isolata dal resto dell'occupazione di base del processo visto l'azzeramento fatto all'inizio).

Per l'algoritmo di stampa ricorsivo abbiamo anche misurato la profondità dello stack di ricorsione: viene catturato l'indirizzo di una variabile locale (`base_marker`) in due momenti, appena prima della chiamata e nel caso base della ricorsione più profonda, prendendo la loro differenza si ottiene una stima in byte dello spazio di stack effettivamente consumato dalle chiamate ricorsive non in coda.

Altre librerie utili che sono state usate sono:

- `<iostream>`, `<iomanip>` `<fstream>` per visualizzare e gestire l'output su linea di comando e su file di testo,
- `<string>`, `<vector>` per le omonime strutture dati necessarie
- `<algorithm>` per sfruttare funzioni generiche già implementate come `max()`, `swap()`.
- `<cstdint>` per convertire i puntatori ad indirizzi di memoria in numeri interi su cui possiamo fare le operazioni necessarie.

Il compilatore GCC può far girare il codice con una serie di ottimizzazioni su cicli, ricorsioni, accessi in memoria... ci sono diversi livelli di ottimizzazione che si possono impostare: O0 traduce in istruzioni macchina il codice così com'è, O1 fa poche ottimizzazioni, O2 fa molte più trasformazioni del codice).

Un livello di ottimizzazione più basso permette di valutare in modo più realistico le variazioni, mentre un ottimizzatore migliore tende a velocizzare soprattutto il codice meno efficiente. Abbiamo usato per le misure il livello O1 perchè permette di attivare/disattivare manualmente l'ottimizzazione sulle chiamate ricorsive (e funzioni sorelle) in coda usando l'opzione `-foptimize-sibling-calls / -fno-optimize-sibling-calls`.

Tutte le misure con compilatore ottimizzante, e non, sono state effettuate in questo modo.

La tabella **c** che contiene le lunghezze delle LCS è stata implementata come matrice di interi (a 4 byte in C++), mentre la tabella **b** che contiene le direzioni è stata implementata come matrice di caratteri (1 byte in C++) sostituendo alle frecce delle lettere che indichino le direzioni.

Ci sono varianti che non usano la tabella **b**, ma nella nostra implementazione questa è in char quindi occupa 1/4 della memoria della tabella **c**.

Per la misurazione dei tempi di esecuzione abbiamo definito una sequenza di interi da interpretare come la lunghezza delle stringhe ed abbiamo realizzato una breve funzione che generi una stringa casuale della lunghezza data usando caratteri di un alfabeto dato (la generazione della stringa non rientra tra i tempi misurati per il calcolo della LCS).

2.1. Misurazioni

Esperimento con stringhe di uguale lunghezza

Nelle prime soluzioni implementate con il codice abbiamo considerato le due stringhe di una stessa lunghezza n , perciò la complessità dell'algoritmo di creazione diventa $\Theta(n^2)$ e quella dell'algoritmo di stampa diventa $O(2n) = O(n)$.

Tabella 1: Risultati del primo esperimento con $n = m$

Dimensione ($m=n$)	Compilatore	Tempo LCS (ms)	Tempo PRINT- LCS (ms)	RAM Heap (KB)	RAM Stack (KB)
10	non ottimiz.	0,10	< 0,01	256	1,45
	ottimizzante	0,08	< 0,01	256	0,80
50	non ottimiz.	0,24	< 0,01	352	6,70
	ottimizzante	0,17	< 0,01	348	3,54
100	non ottimiz.	0,33	0,02	404	12,51
	ottimizzante	0,36	< 0,01	404	7,04
200	non ottimiz.	1,18	0,02	712	25,63
	ottimizzante	1,66	0,02	700	13,93
300	non ottimiz.	2,09	0,06	1240	38,10
	ottimizzante	2,67	0,04	1252	20,82
400	non ottimiz.	3,44	0,06	1860	50,29
	ottimizzante	3,97	0,04	1860	28,04

Dimensione (m=n)	Compilatore	Tempo LCS (ms)	Tempo PRINT- LCS (ms)	RAM Heap (KB)	RAM Stack (KB)
500	non ottimiz.	6,68	0,07	2424	63,13
	ottimizzante	6,66	0,04	2428	35,26
600	non ottimiz.	9,48	0,14	2536	76,35
	ottimizzante	9,84	0,05	2528	42,37
700	non ottimiz.	14,93	0,08	3176	89,85
	ottimizzante	14,67	0,10	3176	49,59
800	non ottimiz.	14,15	0,11	4932	102,04
	ottimizzante	14,27	0,11	4932	56,80
900	non ottimiz.	19,30	0,13	5088	113,29
	ottimizzante	22,83	0,05	5084	62,05
1000	non ottimiz.	21,83	0,12	6768	126,13
	ottimizzante	20,01	0,09	6776	70,91
2000	non ottimiz.	97,25	0,22	21972	253,54
	ottimizzante	81,27	0,22	22268	143,43
3000	non ottimiz.	187,91	0,36	48212	381,13
	ottimizzante	176,23	0,28	48128	213,10
4000	non ottimiz.	375,58	0,42	84692	505,73
	ottimizzante	312,38	0,34	84196	282,12
5000	non ottimiz.	530,05	0,62	125552	632,01
	ottimizzante	662,71	0,43	125560	356,82
6000	non ottimiz.	696,01	0,88	181780	756,98
	ottimizzante	819,23	0,63	181872	428,46
7000	non ottimiz.	1093,71	0,70	250128	885,60
	ottimizzante	1278,38	0,75	250368	498,90
8000	non ottimiz.	1380,12	0,82	341300	1010,85
	ottimizzante	1655,53	0,64	341492	571,30
9000	non ottimiz.	1894,07	1,68	430224	1139,76
	ottimizzante	2149,64	0,71	430092	641,52
10000	non ottimiz.	2573,93	1,50	529460	1264,45
	ottimizzante	2406,45	0,68	529972	718,52
15000	non ottimiz.	4094,61	1.81	1189716	1894,73
	ottimizzante	4360,77	1.57	1187232	1070,38

Esperimento n variabile, m costante

Questa serie di esperimenti avevano lo scopo principale di analizzare la complessità spaziale della pila dei processi della funzione PRINT-LCS con compilatore ottimizzante e non, per verificare l'ipotesi riportata nel capitolo 1.4.

Si nota come l'andamento del «Tempo LCS (ms)» segue l'andamento $O(n)$, in linea con la teoria per entrambe le tipologie di compilatori, infatti $O(m \cdot n) = O(n)$ con m costante.

Più rilevante è il comportamento dell'occupazione in memoria della pila dei contesti «RAM Stack (KB)».

L'ipotesi (nel capitolo 1.4) era:

- Complessità spaziale della pila dei contesti = $O(n + m)$ per compilatore non ottimizzante
- Complessità spaziale della pila dei contesti = $O(\min(n, m))$ per compilatore ottimizzante

Per i valori di $400 \leq n \leq 10000$ lo spazio occupato dallo stack rimane costante se è utilizzato il compilatore ottimizzante, mentre varia se si utilizza un compilatore non ottimizzante.

I risultati sono in linea con l'ipotesi proposta: nonostante sia presente una chiamata non in coda nell'algoritmo, l'utilizzo di un compilatore ottimizzante comporta comunque delle miglie per quanto riguarda l'occupazione dello spazio in memoria.

Tabella 2: Risultati del secondo esperimento con n variabile, m costante

Dimensione ($n, m=100$)	Compilatore	Tempo LCS (ms)	Tempo PRINT- LCS (ms)	RAM Heap (KB)	RAM Stack (KB)
10	non ottimiz.	0,09	< 0,01	216	2,57
	ottimizzante	0,09	< 0,01	216	1,24
50	non ottimiz.	0,25	0,01	412	8,57
	ottimizzante	0,28	< 0,01	424	4,63
100	non ottimiz.	0,36	0,01	500	12,41
	ottimizzante	0,47	0,01	512	7,04
200	non ottimiz.	0,68	0,02	636	20,29
	ottimizzante	0,96	0,01	608	9,45
300	non ottimiz.	0,75	0,02	740	27,13
	ottimizzante	1,90	0,03	736	10,65
400	non ottimiz.	2,14	0,06	868	35,66
	ottimizzante	2,65	0,01	880	11,09
500	non ottimiz.	1,45	0,03	1060	43,63
	ottimizzante	2,20	0,02	1072	11,09
600	non ottimiz.	2,42	0,03	1176	41,76
	ottimizzante	3,06	0,02	1208	11,09

Dimensione (n, m=100)	Compilatore	Tempo LCS (ms)	Tempo PRINT- LCS (ms)	RAM Heap (KB)	RAM Stack (KB)
700	non ottimiz.	2,32	0,03	1340	41,10
	ottimizzante	2,90	0,01	1352	11,09
800	non ottimiz.	2,54	0,03	1388	37,91
	ottimizzante	2,79	0,02	1384	11,09
900	non ottimiz.	4,44	0,03	2024	44,20
	ottimizzante	4,05	0,03	2124	11,09
1000	non ottimiz.	2,85	0,02	2148	35,20
	ottimizzante	4,95	0,01	2240	11,09
2000	non ottimiz.	6,43	0,03	3424	47,48
	ottimizzante	8,36	0,01	3264	11,09
3000	non ottimiz.	10,37	0,04	3700	36,70
	ottimizzante	11,43	0,01	3780	11,09
4000	non ottimiz.	11,78	0,03	4592	38,48
	ottimizzante	12,59	0,02	4780	11,09
5000	non ottimiz.	15,95	0,04	6020	41,48
	ottimizzante	16,67	0,02	6104	11,09
6000	non ottimiz.	23,22	0,02	6800	37,91
	ottimizzante	20,65	0,02	6896	11,09
7000	non ottimiz.	19,44	0,03	7828	38,10
	ottimizzante	34,40	0,02	7912	11,09
8000	non ottimiz.	18,68	0,03	8424	38,95
	ottimizzante	41,06	0,02	8500	11,09
9000	non ottimiz.	24,79	0,02	9164	38,20
	ottimizzante	30,18	0,01	9232	11,09
10000	non ottimiz.	24,26	0,02	10036	39,13
	ottimizzante	37,04	0,01	10088	11,09
15000	non ottimiz.	37,38	0,03	14232	38.76
	ottimizzante	35.55	0.01	14224	11.09

Nella tabella successiva sono riportati i risultati relativi ai seguenti casi limite:

- **Stringhe senza LCS, $LCS = 0$:** questo è il caso pessimo per LCS-PRINT per la complessità spaziale con compilatore non ottimizzante, mentre è il caso migliore per la complessità spaziale con compilatore ottimizzante. I dati riportati sono in linea con la teoria. Il tempo di esecuzione di LCS-PRINT e l'occupazione di spazio con compilatore non ottimizzante non differiscono troppo dal caso con stringhe generate randomicamente. Invece la differenza tra il caso ottimale e quello generato randomicamente è notevole, si passa da utilizzare 468.88 KB a 0.15KB perché tutte le chiamate sono chiamate ricorsive in coda.
- **Stringhe identiche, $LCS = n$:** nell'esecuzione di LCS-PRINT in questo caso tutte le chiamate non sono ricorsive in coda, quindi non sono presenti miglioramenti da segnalare.

In entrambi questi casi il tempo di esecuzione dell'algoritmo LCS non varia dato che la complessità spaziale è esattamente $\Theta(n \cdot m)$.

- **Stringhe di lunghezza diversa ma con un singolo carattere:** questo caso non riporta risultati degni di nota, il tempo di esecuzione e lo spazio occupato diminuiscono rispetto al caso precedente per la riduzione di m da 5000 a 200. Questo caso è stato ideato per controllare che la funzione fosse corretta anche in questo caso limite.

Tabella 3: Risultati esperimento 3, casi limite

Esperimento	Compilatore	Tempo LCS (ms)	Tempo PRINT-LCS (ms)	RAM Heap (KB)	RAM Stack (KB)
LCS = 0 ($n = m = 5000$)	non ottimiz.	407,26	0,35	135660	468,88
	ottimizzante	564,36	0,16	134976	0,15
LCS = n ($n = m = 5000$)	non ottimiz.	420,98	0,40	133940	468,88
	ottimizzante	471,18	0,46	133796	547,02
$ \Sigma = 1$ ($n = 5000, m = 200$)	non ottimiz.	17,52	0,02	9284	18,88
	ottimizzante	30,82	0,01	9276	22,02

In tutte le tabelle si può notare come il tempo di esecuzione di LCS sia maggiore se viene utilizzato il compilatore ottimizzante, mentre per PRINT-LCS il tempo viene diminuito.

Questo comportamento è causato dalla tipologia di funzione:

- LCS è una funzione iterativa, non trae vantaggio da un compilatore ottimizzante.
- LCS-PRINT è un funzione ricorsiva con alcune chiamate in coda, il compilatore ottimizzante permette di tradurre queste chiamate in dei cicli a livello macchina, riducendo le operazioni necessarie per la gestione della pila dei contesti delle funzioni.

2.2. Visualizzazione dei dati

I grafici presentati sono stati realizzati con MATLAB, i dati che abbiamo misurato vengono rappresentati come punti su un piano cartesiano ed il loro andamento visualizzato tramite una retta/parabola realizzata con il comando `polyfit` che trova i coefficienti del polinomio di grado specificato che meglio approssima i dati usando un'approssimazione ai minimi quadrati, una volta calcolati i polinomi sono stati valutati in diversi punti tramite il comando `polyval` e poi visualizzati sullo stesso grafico dei dati.

I grafici hanno sull'asse delle ascisse la dimensione del problema (lunghezza di una o entrambe le stringhe) che va da 100 a 10000 in una scala lineare; i primi quattro grafici visualizzano dati contenuti nella Tabella 1, mentre l'ultimo usa dati contenuti nella Tabella 2.

Figura 1: Tempo di esecuzione dell'algoritmo LCS-LENGTH in funzione della lunghezza delle stringhe ($m = n$), con andamento quadratico, da teoria $\Theta(m \cdot n)$

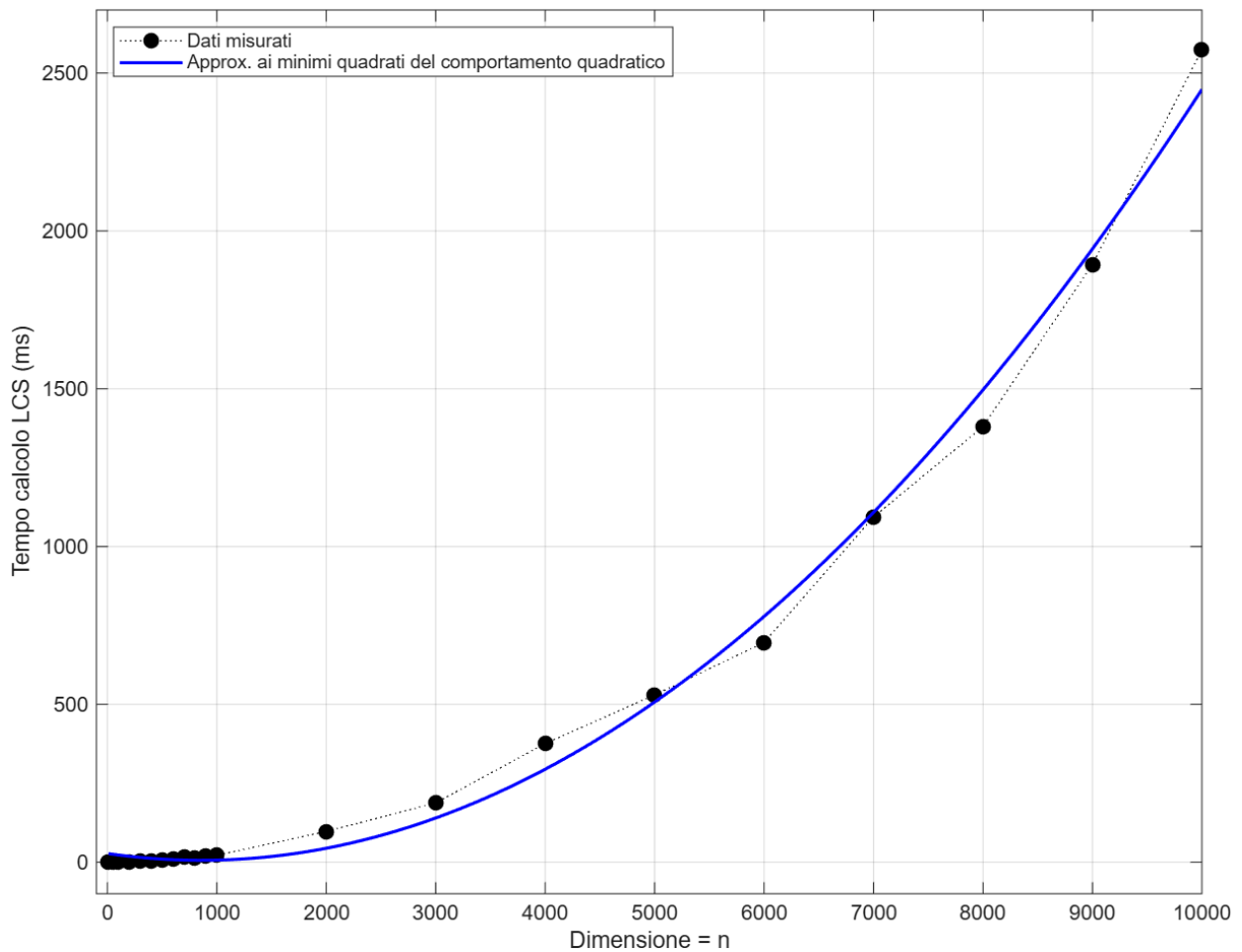
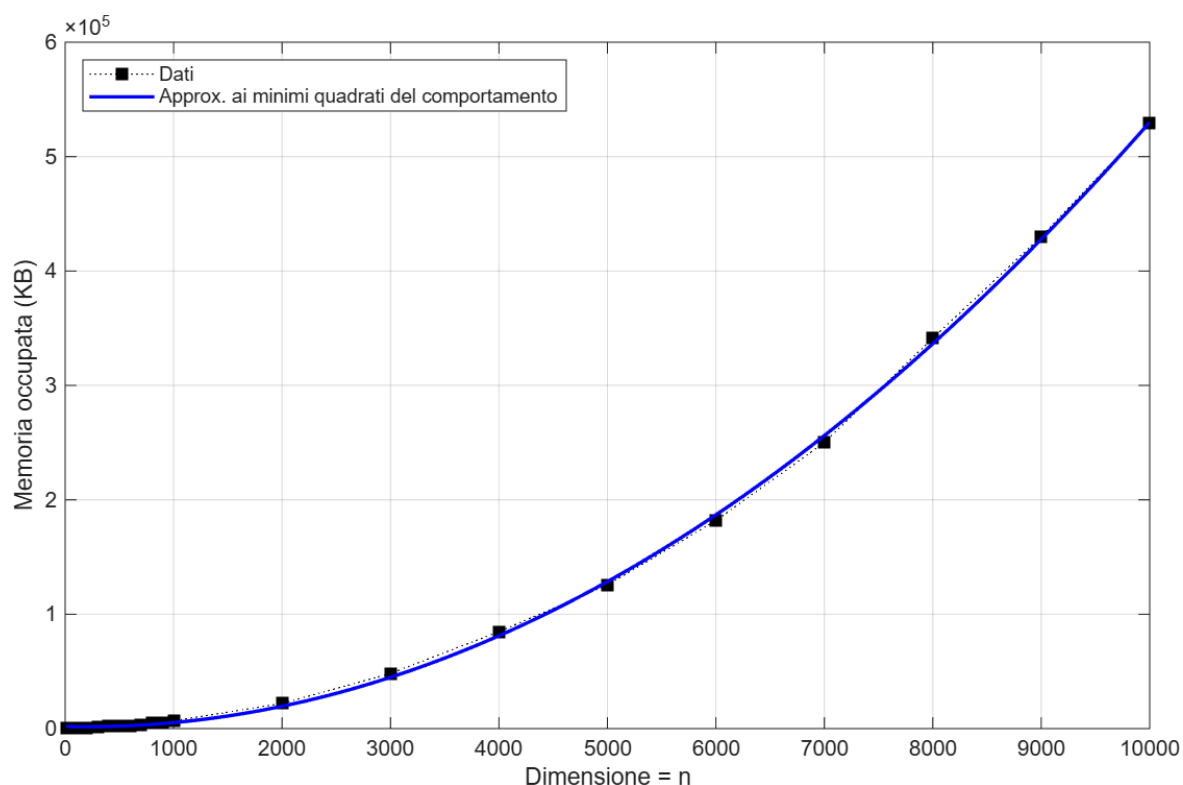


Figura 2: Memoria necessaria per l'esecuzione dell'algoritmo LCS-LENGTH in funzione della lunghezza delle stringhe ($m = n$), con andamento quadratico, da teoria $\Theta(m \cdot n)$



Le due complessità di LCS-LENGTH sono entrambe $\Theta(m \cdot n)$ (con $m = n$, $\Theta(n^2)$).

Vediamo dai grafici che le misurazioni effettuate rispecchiano fedelmente la teoria, il che è un risultato abbastanza atteso dato che il limite asintotico vale sia superiormente che inferiormente, perciò trovare una parabola che approssimi i dati dovrebbe essere sempre possibile.

Le misurazioni effettuate per PRINT-LCS sembrano essere leggermente meno aderenti ai risultati teorici, potrebbe essere dovuto al fatto che la complessità temporale dell'algoritmo di stampa ha solo un limite asintotico superiore perciò non è garantito di trovare una retta che approssima esattamente il comportamento.

I grafici sono riportati con la stessa scala in modo che si possa vedere il miglioramento temporale dovuto all'uso del compilatore ottimizzante (ogni volta che si evita di allocare un nuovo contesto si risparmia anche del tempo, riscrivere un paio di variabili è sicuramente più veloce).

Non è stata ripetuta la misurazione sulla memoria heap per l'algoritmo di stampa dato che la complessità era identica a quella dell'algoritmo di calcolo già visualizzato in precedenza.

Figura 3: Tempo di esecuzione dell'algoritmo PRINT-LCS in funzione della lunghezza delle stringhe ($m = n$), senza compilatore ottimizzante, da teoria $O(m + n)$

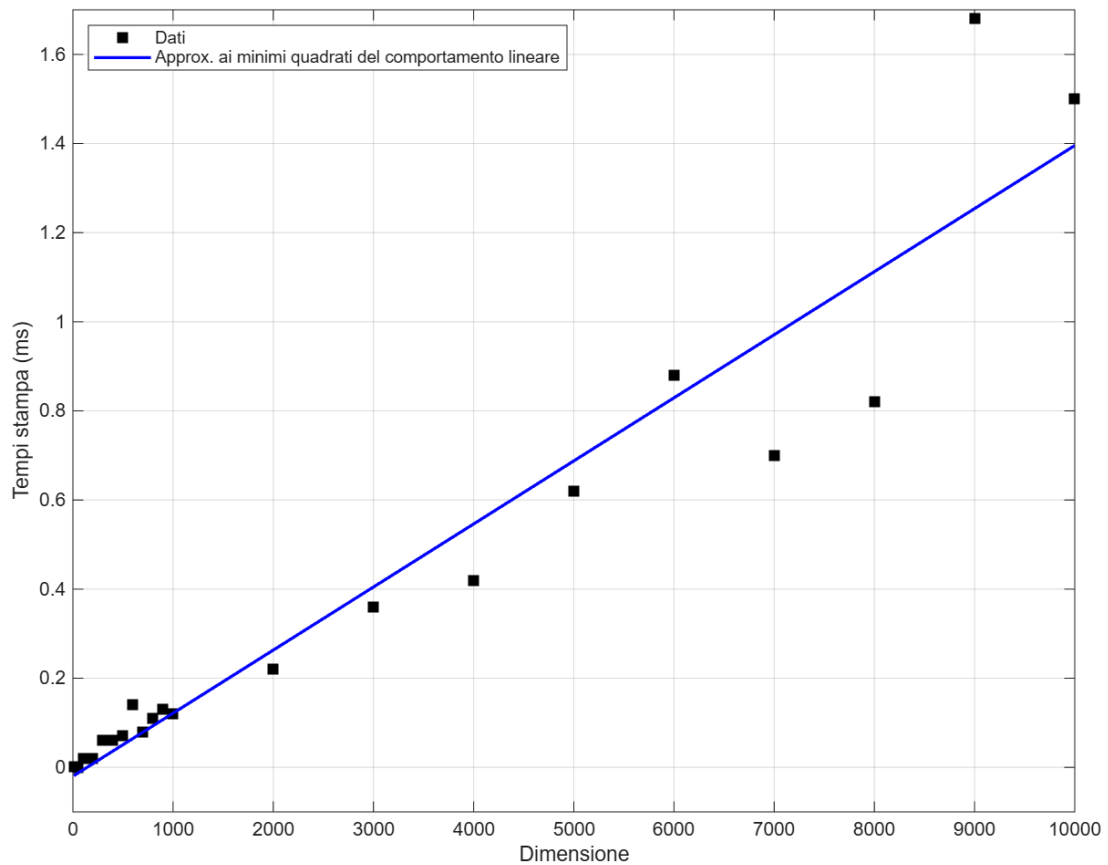


Figura 4: Tempo di esecuzione dell'algoritmo PRINT-LCS in funzione della lunghezza delle stringhe ($m = n$), con compilatore ottimizzante, da teoria $O(m + n)$

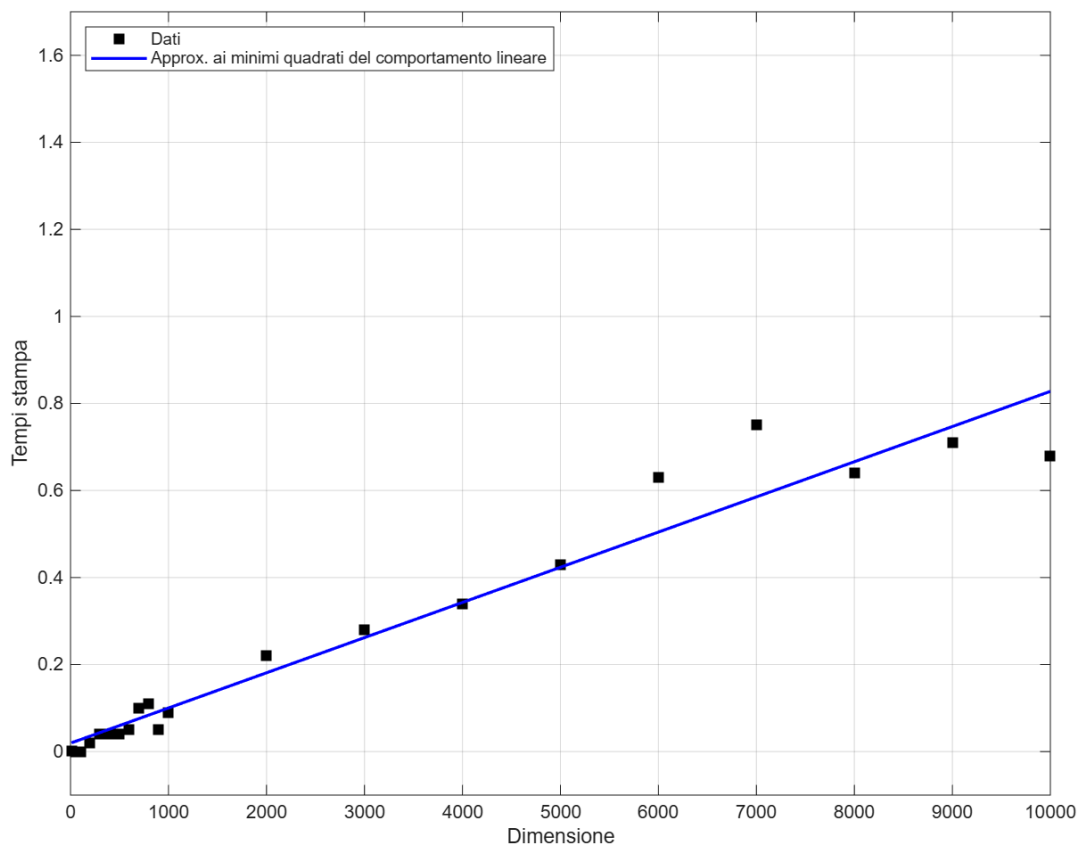
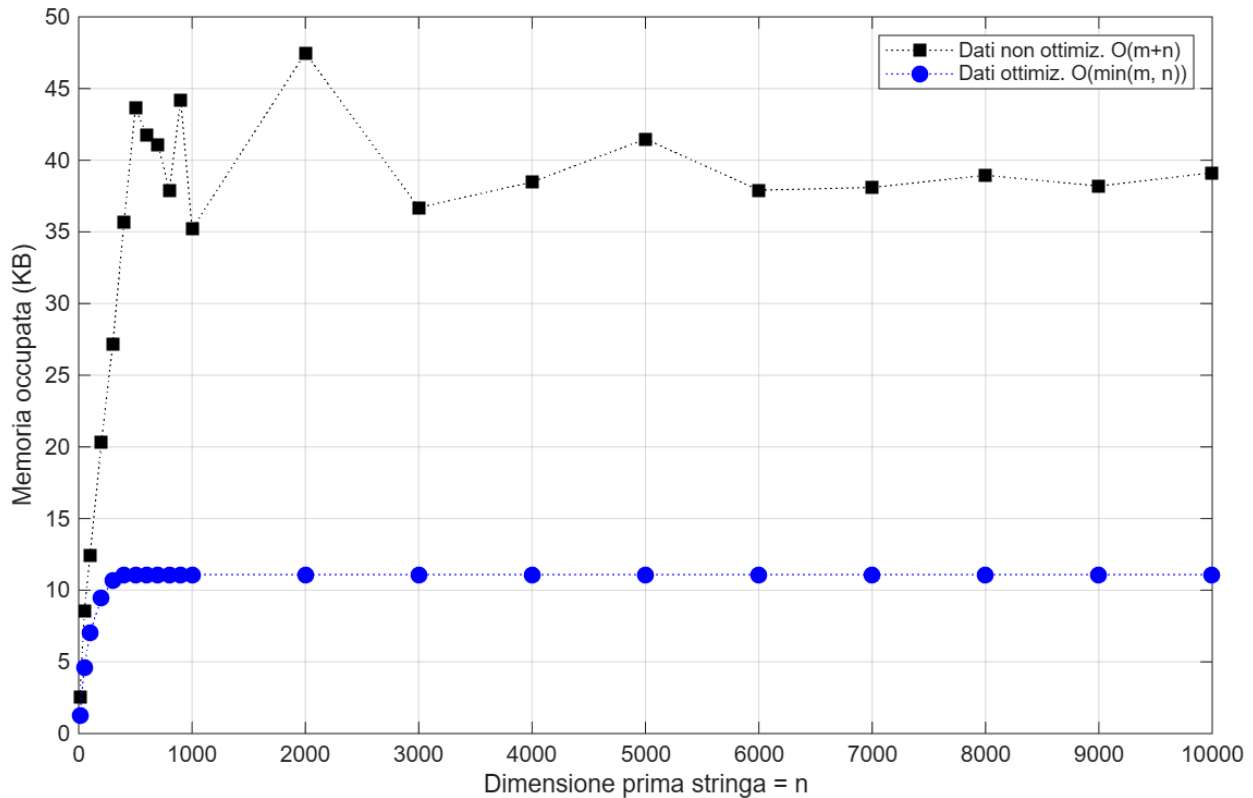


Figura 5: Memoria occupata dallo stack di ricorsione dell'algoritmo **PRINT-LCS** in funzione della lunghezza della prima stringa n ($m = 100$ fissato), in nero le misurazioni con compilatore non ottimizzante, in blu con compilatore ottimizzante



Abbiamo voluto misurare praticamente se la differenza che avevamo teorizzato sullo stack di ricorsione di **PRINT-LCS** esistesse davvero, così pare, con il compilatore ottimizzante si hanno valori costanti da una certa dimensione in poi, coerentemente con la nostra valutazione $O(\min(m, n))$ risultano più bassi di quelli ottenuti con il compilatore non ottimizzante.

3. Varianti dell'algoritmo **LCS-LENGTH**

Le miglirie qui presentate riguardano solo l'algoritmo **LCS-LENGTH** che ha lo scopo di calcolare la lunghezza della LCS, nello pseudocodice riportato in precedenza viene costruita anche la matrice **b** molto comoda per ricostruire la LCS e visualizzarla, le successive varianti non si preoccupano di farlo, sono alternative strettamente limitate ad **LCS-LENGTH**.

Versione 2

Ci si può accorgere che all'interno del ciclo esterno della funzione **LCS-LENGTH** vengono usate solo le righe di indice **i**, **i-1**, cioè ad ogni iterazione basta conoscere solo i valori contenuti nella riga precedente (tutte le righe già completate ad eccezione di quella immediatamente precedente non sono più utili al fine di calcolare il risultato finale desiderato).

Quindi per calcolare la lunghezza della LCS non serve mantenere in memoria l'intera matrice $c[0 \dots m, 0 \dots n]$, ma basterà procedere due righe alla volta sovrascrivendole man mano, quando si arriva a calcolare l'ultima riga allora si leggerà l'elemento presente nella sua ultima colonna e quella sarà la lunghezza della LCS.

La memoria richiesta quindi non è più proporzionale alla lunghezza di entrambe le stringhe (che definivano le dimensioni della matrice), ma diventa proporzionale solo alla lunghezza di una delle due stringhe, quella cui lunghezza definisce il numero di colonne n .

Si passa dall'originale spazio necessario di $\Theta(m \cdot n)$ a quello necessario per salvare i contenuti di due righe (implementate come due vettori di interi), ovvero $\Theta(2 \cdot n) = \Theta(n)$, se le due stringhe sono di uguale lunghezza si passa da una crescita di memoria quadratica ad una lineare.

La regola con cui si costruisce la riga successiva rimane comunque la stessa: se i caratteri corrispondono si guarda il valore «diagonale» e si aggiunge 1, altrimenti si prende il massimo tra il valore contenuto nella cella a sinistra e quella sopra. Il numero di operazioni totali da fare non cambia, quindi il tempo richiesto sarà pressoché uguale.

Versione 3

Un'ulteriore alternativa che permette una riduzione, stavolta meno significativa, dell'utilizzo di memoria è quella di utilizzare un solo vettore anziché due.

L'operazione fatta in precedenza infatti non ha bisogno di due copie fisiche separate delle due righe, si può far scorrere una singola riga ed aggiornare ogni cella man mano, i valori precedentemente scritti sono quelli che prima stavano nella riga precedente, i valori nuovi che vengono sovrascritti corrispondono alla riga successiva calcolata.

È lo stesso vettore che man mano viene aggiornato da riga precedente a successiva.

Uno dei 3 casi dell'algoritmo **LCS-LENGTH** richiede il valore "in diagonale" ovvero quello di $c[i-1][j-1]$, questo era perfettamente fattibile con due righe, ma con un solo vettore bisogna attuare un piccolo accorgimento: considerato un elemento del vettore, l'elemento alla sua sinistra viene aggiornato subito prima e quindi andrebbe perso il valore "in diagonale" che ci serve, si risolve semplicemente salvando il valore interessato in una variabile temporanea prima che questo venga sovrascritto.

Chiaramente con questa versione non si ha un miglioramento a livello asintotico, la complessità spaziale rimane $O(n)$, cambiano le costanti moltiplicative dato che si riduce da due vettori ad uno solo con l'aggiunta di un paio di variabili scalari intere.

Abbiamo implementato queste due versioni con un controllo che pone sempre come seconda stringa quella più corta, così il numero delle colonne $n < m$, e i vettori verranno sempre creati con lunghezza n e quindi con il minor numero di celle. Se la prima stringa è più corta della seconda viene chiamata la funzione **swap()** che in un tempo costante scambia i puntatori alle due stringhe. Ci sono poi sempre due cicli annidati che fanno rispettivamente m, n iterazioni, il numero di operazioni complessivo non si riduce ma lo spazio necessario in memoria sì.

3.1. Misurazioni

Verifichiamo sperimentalmente la differenza in termini di memoria utilizzata dalle 3 versioni dell'algoritmo usando istanze di dimensioni notevoli in cui queste differenze possono essere apprezzate.

Tabella 4: Risultati esperimento tempi di calcolo per $m = n$

Dimensione (m = n)	Tempo Versione 1 (ms)	Tempo Versione 2 (ms)	Tempo Versione 3 (ms)
1000	25,82	12,05	13,61
5000	563,55	497,13	266,29
10000	2440,06	1185,37	1284,05
15000	4882,95	2289,49	2479,16
20000	N/A	4424,88	3472,08
25000	N/A	5996,25	5899,61
30000	N/A	8875,23	9076,04
35000	N/A	12898,06	12696,41
40000	N/A	16520,72	14350,36
45000	N/A	20185,39	21365,55
50000	N/A	23468,24	22808,39

Tabella 5: Risultati esperimento memoria RAM Heap tabelle per $m = n$

Dimensione (m = n)	RAM Heap (KB) Versione 1	RAM Heap (KB) Versione 2	RAM Heap (KB) Versione 3
1000	6784	204	184
5000	126512	300	236
10000	529396	376	292
15000	1188412	416	320
20000	N/A	584	356
25000	N/A	532	392
30000	N/A	568	412
35000	N/A	664	488
40000	N/A	696	504
45000	N/A	740	540
50000	N/A	812	584

Tabella 6: Risultati esperimento tempi di calcolo per n variabile e $m = 100$

Dimensione (n, m = 100)	Tempo Versione 1 (ms)	Tempo Versione 2 (ms)	Tempo Versione 3 (ms)
1000	2,78	0,88	1,66
5000	15,21	4,64	4,13
10000	33,55	9,02	11,52
20000	47,45	16,01	15,49
30000	75,12	23,97	21,87
40000	103,27	36,51	39,14
50000	125,32	40,32	48,47

Tabella 7: Risultati esperimento memoria RAM Heap tabelle per n variabile e $m = 100$

Dimensione (n, m = 100)	RAM Heap (KB) Versione 1	RAM Heap (KB) Versione 2	RAM Heap (KB) Versione 3
1000	1744	208	176
5000	6740	204	184
10000	10872	208	176
20000	17844	220	184
30000	25728	232	200
40000	32028	240	204
50000	41332	252	216

Tutti i risultati riportati in queste tabelle sono stati eseguiti con compilatore ottimizzato, anche sulle nuove versioni non erano presenti differenze significative usando diversi compilatori dato che non sono state introdotte ricorsioni in coda.

I risultati riportati con N/A indicano le esecuzioni in cui il programma si è interrotto per overflow di memoria.

Come ipotizzato dalla teoria si ottiene un grande miglioramento nel consumo di memoria passando dalla versione 1 alla versione 2, visibile in particolare dalla Tabella 5 dove si vede la grandissima differenza tra l'andamento quadratico del primo algoritmo e quello lineare delle nuove versioni.

Tra la versione 2 e 3 invece, come prevedibile, le differenze sono nettamente minori, ma comunque il miglioramento in termini di memoria è presente, per quanto riguarda i tempi di esecuzione in media si equivalgono.

Dalla Tabella 7 si vede l'accorgimento che abbiamo fatto per garantire che i vettori abbiano sempre dimensione pari alla stringa più corta tra le due: nonostante la lunghezza n della stringa maggiore aumenti, la minore rimane di dimensione fissata m e quindi durante tutte le esecuzioni i vettori avranno le stesse dimensioni e la memoria richiesta per il loro allocamento rimane uguale.

Passando alle versioni migliorate si ottengono anche miglioramenti nel tempo di esecuzione, a livello asintotico nella teoria non ci sono riduzioni perché rimangono presenti i due cicli annidati con lo stesso numero di iterazioni, evidentemente nella pratica cambiano le costanti moltiplicative.

4. Note

Le informazioni necessarie per la compilazione e le esecuzioni degli algoritmi presentati in questa relazione sono riportate nel `README.md` sulla pagina *GitHub* relativa al progetto, accessibile tramite il seguente link:

https://github.com/Jonz537/Progetto_Algoritmi_e_Strutture_Dati/blob/main/README.md

Nella pagina *GitHub* relativa al progetto (https://github.com/Jonz537/Progetto_Algoritmi_e_Strutture_Dati) è possibile anche accedere ai file di output nella loro completezza:

- `results_tail.txt`: risultati dei test relativi al compilatore ottimizzante.
- `results_no_tail.txt`: risultati dei test relativi al compilatore non ottimizzante.

5. Bibliografia:

- Pagina Wikipedia: https://en.wikipedia.org/wiki/Longest_common_subsequence
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein;
Introduzione agli algoritmi e strutture dati; Quarta Edizione; McGraw-Hill; 2023
- 1143 - Longest Common Subsequence; Leetcode:
<https://leetcode.com/problems/longest-common-subsequence/description/>
- <https://www.geeksforgeeks.org/dsa/longest-common-subsequence-dp-4/>